Autor:
Manuel Eugenio Morocho Cayamcela, PhD

Prueba práctica:

Clasificación de Pinguinos de la Isla Palmer con un perceptrón multicapa (MLP)

Objetivo y alcance del trabajo:

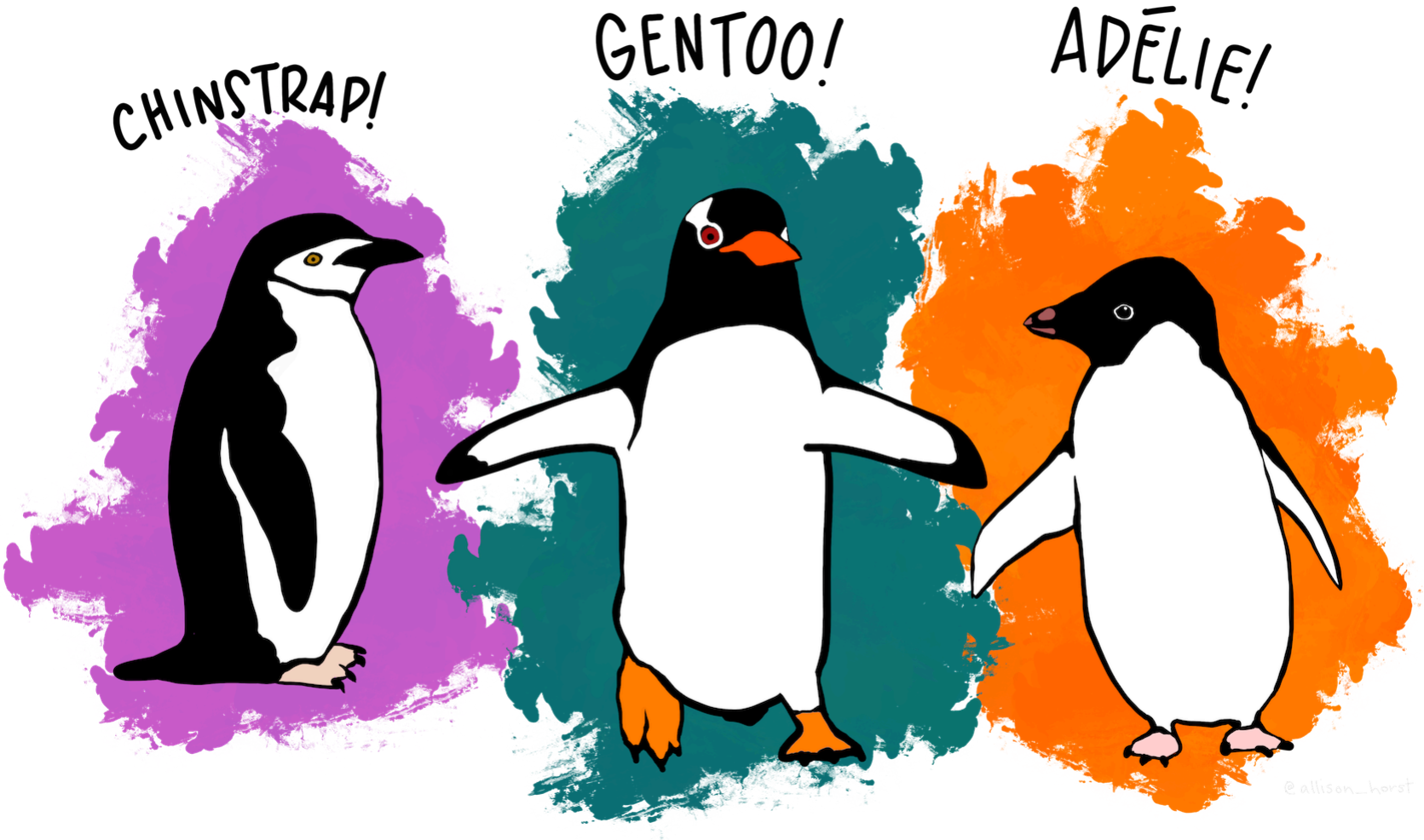
Objetivo:

El objetivo de este Jupyter Notebook es desarrollar y evaluar un modelo de clasificación utilizando un Perceptrón Multicapa (MLP). El modelo se entrenará utilizando un conjunto de datos de entrenamiento y luego se utilizará para hacer predicciones en un conjunto de datos de prueba.

Alcance:

Este Jupyter Notebook cubrirá los siguientes aspectos:

1. Importación de las bibliotecas necesarias.
2. Carga y exploración inicial de los datos.
3. Preprocesamiento básico de los datos.
4. División de los datos en conjuntos de entrenamiento y prueba.
5. Creación de un MLPClassifier.
6. Entrenamiento del MLPClassifier con el conjunto de entrenamiento.
7. Uso del MLPClassifier entrenado para hacer predicciones en el conjunto de prueba.
8. Evaluación del rendimiento del modelo.



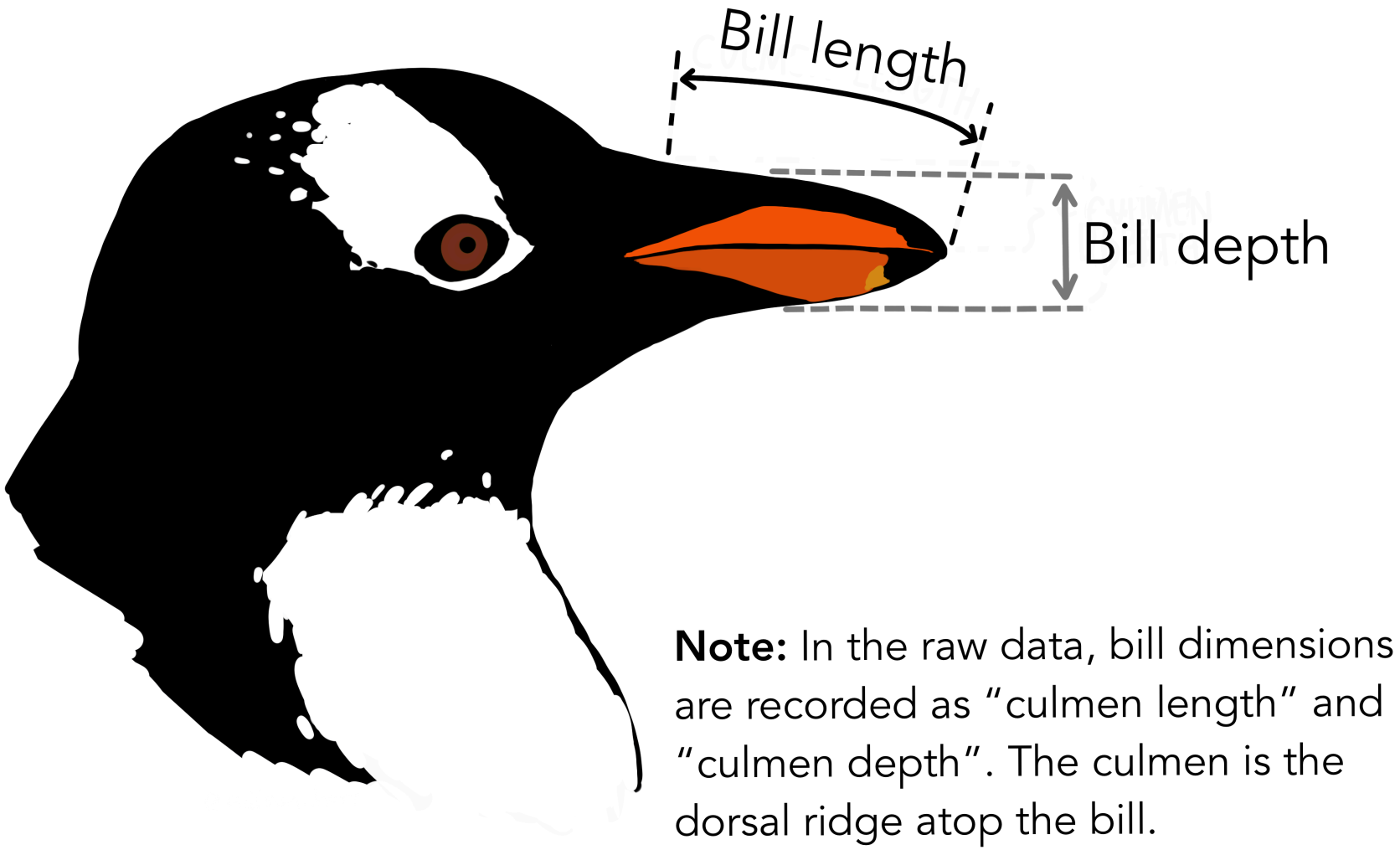
El conjunto de datos de Palmer Penguins fue introducido en 2020 por Allison Horst, Alison Hill y Kristen Gorman. Este conjunto de datos es a menudo utilizado para tareas de clasificación en aprendizaje automático.

El conjunto de datos contiene 344 pingüinos observados en tres diferentes islas del archipiélago de Palmer, en la Antártida. Los datos fueron recogidos por el proyecto Palmer Long Term Ecological Research y la Fundación Nacional para la Ciencia de los Estados Unidos.

Cada pingüino se clasifica en una de las tres especies: Adelie, Chinstrap y Gentoo. Para cada pingüino, el conjunto de datos incluye siete características:

1. Especie: Adelie, Chinstrap, Gentoo.
2. Isla: Torgersen, Biscoe, Dream.
3. Longitud del pico (mm).
4. Profundidad del pico (mm).
5. Longitud de la aleta (mm).
6. Masa corporal (g).
7. Sexo: masculino, femenino.

El objetivo principal al trabajar con este conjunto de datos es a menudo construir un modelo de clasificación que pueda predecir la especie de un pingüino basándose en las características que se tienen en la base de datos.



Fase I: Carga y procesamiento de la base de datos

In [1]:
```
# Importamos las librerías necesarias
import numpy as np
import matplotlib.pyplot as plt
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

In [2]:
```
# Cargamos la base de datos 'palmer_penguins.csv' en un DataFrame de Pandas de nombre 'pinguinos'
pinguinos = pd.read_csv('palmer_penguins.csv')

# Mostremos las primeras 5 filas de la base de datos
pinguinos.head()
```

Out[2]:

	species	island	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	MALE
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	FEMALE
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	FEMALE
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	FEMALE

In [3]:
```
# Usamos el método 'isnull()' para identificar los valores nulos y el método 'sum()' para sumarlos
pinguinos.isnull().sum()
```

Out[3]:
```
species              0
island               0
culmen_length_mm     2
culmen_depth_mm      2
flipper_length_mm    2
body_mass_g          2
sex                 10
dtype: int64
```

In [4]:
```
# Usamos el método 'dropna()' para eliminar las filas con valores nulos del DataFrame 'pinguinos'
pinguinos = pinguinos.dropna()
```

```
# Usamos el método 'head()' para mostrar las primeras 5 filas de la base de datos
pinguinos.head()
```

Out[4]:

	species	island	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	MALE
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	FEMALE
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	FEMALE
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	FEMALE
5	Adelie	Torgersen	39.3	20.6	190.0	3650.0	MALE

In [5]:
```
# Usamos el método 'unique()' para encontrar los valores únicos en la columna 'sex'
pinguinos['sex'].unique()
```

Out[5]:
```
<StringArray>
['MALE', 'FEMALE', '.']
Length: 3, dtype: str
```

In [6]:
```
# Eliminamos las filas con el valor '.' en la columna 'sex' usando el método de filtrado por comparación negada '!='
pinguinos = pinguinos[pinguinos['sex'] != '.']
```

Fase II: Preparamos los datos para ser usados en un perceptrón multicapa (MLP)

In [7]:
```
# Aislamos las variables predictoras que son todas menos la variable objetivo
# Usamos el método 'drop()' para eliminar la columna 'species' del DataFrame 'pinguinos'
X = pinguinos.drop('species', axis=1)

# Convertimos las variables categóricas en variables numéricas mediante el método 'LabelEncoder()'
# Usamos el método 'apply()' para aplicar la función 'LabelEncoder().fit_transform' a cada columna de 'X'
X = X.apply(LabelEncoder().fit_transform)

# Mostremos las primeras 5 filas de la base de datos
X.head()
```

Out[7]:

	island	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body_mass_g	sex
0	2	41	56	5	30	1
1	2	44	43	10	32	0
2	2	50	49	19	11	0
4	2	21	62	17	18	0
5	2	43	73	14	26	1

In [8]:
```
# Aislamos la variable objetivo 'species' en la variable 'y'
y = pinguinos['species']

# Mostramos las primeras 5 filas de la base de datos
y.head()
```

Out[8]:
```
0    Adelie
1    Adelie
2    Adelie
4    Adelie
5    Adelie
Name: species, dtype: str
```

In [9]:
```
# Dividimos los datos 'X' y 'y' en entrenamiento y prueba con el método 'train_test_split'
# Usamos el 0.2 (20%) de los datos para prueba
# Guardamos los datos de entrenamiento en las variables 'X_train' y 'y_train'
# Guardamos los datos de prueba en las variables 'X_test' y 'y_test'
# (random_state=42 para que los resultados sean reproducibles)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Fase III: Clasificación de pinguinos con un perceptrón multicapa (MLP)

In [10]:
```
# Creamos un clasificador de tipo MLPClassifier con 2 capas ocultas de 100 neuronas cada una
# Usamos la función de activación 'relu' y el optimizador 'adam'
# (random_state=42 para que los pesos sean reproducibles y coincidan con el Excel)
mlp = MLPClassifier(hidden_layer_sizes=(100, 100), activation='relu', solver='adam', random_state=42)

# Usamos el método 'fit()' para entrenar el clasificador
```

```
# Entrenamos el clasificador con los datos de entrenamiento 'X_train' y 'y_train'
mlp.fit(X_train, y_train)

# Predecimos las etiquetas de los datos de prueba 'X_test' usando el método 'predict()' del clasificador entrenado
y_pred = mlp.predict(X_test)
y_pred
```

Out[10]:
```
array(['Adelie', 'Gentoo', 'Adelie', 'Chinstrap', 'Adelie', 'Gentoo',
       'Gentoo', 'Chinstrap', 'Chinstrap', 'Chinstrap', 'Adelie',
       'Adelie', 'Gentoo', 'Adelie', 'Gentoo', 'Adelie', 'Adelie',
       'Chinstrap', 'Adelie', 'Gentoo', 'Adelie', 'Adelie', 'Gentoo',
       'Chinstrap', 'Adelie', 'Adelie', 'Gentoo', 'Gentoo', 'Chinstrap',
       'Gentoo', 'Chinstrap', 'Gentoo', 'Adelie', 'Adelie', 'Gentoo',
       'Gentoo', 'Chinstrap', 'Gentoo', 'Adelie', 'Adelie', 'Adelie',
       'Adelie', 'Chinstrap', 'Chinstrap', 'Adelie', 'Adelie', 'Gentoo',
       'Adelie', 'Adelie', 'Gentoo', 'Adelie', 'Gentoo', 'Gentoo',
       'Adelie', 'Adelie', 'Gentoo', 'Chinstrap', 'Adelie', 'Chinstrap',
       'Chinstrap', 'Gentoo', 'Gentoo', 'Gentoo', 'Adelie', 'Adelie',
       'Gentoo', 'Adelie'], dtype='<U9')
```

In [11]:
```
# Calculamos la precisión del modelo de clasificación 'mlp' con el método 'accuracy_score'
# Usamos los datos 'y_test' y 'y_pred' como argumentos de la función 'accuracy_score'
# Guardamos el resultado en la variable 'accuracy'
accuracy = accuracy_score(y_test, y_pred)
accuracy
```

Out[11]:
```
0.9850746268656716
```

In [12]:
```
# Mostramos la matriz de confusión del modelo de clasificación 'mlp' con el método 'confusion_matrix'
# Usamos los datos 'y_test' y 'y_pred' como argumentos de la función 'confusion_matrix'
confusion_matrix(y_test, y_pred)
```

Out[12]:
```
array([[30,  1,  0],
       [ 0, 13,  0],
       [ 0,  0, 23]])
```

In [13]:
```
# Imprimimos el reporte de clasificación del modelo de clasificación 'mlp' con el método 'classification_report'
# Usamos los datos 'y_test' y 'y_pred' como argumentos de la función 'classification_report'
# Imprimimos el reporte de clasificación en la consola con la función 'print'
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

      Adelie       1.00      0.97      0.98        31
   Chinstrap       0.93      1.00      0.96        13
      Gentoo       1.00      1.00      1.00        23

    accuracy                           0.99        67
   macro avg       0.98      0.99      0.98        67
weighted avg       0.99      0.99      0.99        67
```

In [14]:
```python
# Hacemos una prueba de predicción de especie con valores de un nuevo pinguino (no visto por el modelo)
# Creamos una lista con los valores de las variables predictoras del nuevo pinguino
# Puedes usar estos valores de ejemplo o cambiarlos por los valores aleatorios de un nuevo pinguino:
#   island = 1 (Biscoe)
#   culmen_length_mm = 45
#   culmen_depth_mm = 58
#   flipper_length_mm = 11
#   body_mass_g = 22
#   sex = 0 (FEMALE)
# Guardamos la lista en la variable 'nuevo_pinguino'
nuevo_pinguino = [[1, 45, 58, 11, 22, 0]]

# Hacemos una predicción de la especie del 'nuevo_pinguino' con el método 'predict' del modelo 'mlp'
# Guardamos la predicción en la variable 'nueva_prediccion'
nueva_prediccion = mlp.predict(nuevo_pinguino)
nueva_prediccion
```

```
C:\Users\Usuario\Downloads\Actividad 7\.venv\Lib\site-packages\sklearn\utils\validation.py:2830: UserWarning: X does not have vali
d feature names, but MLPClassifier was fitted with feature names
  warnings.warn(
```

Out[14]:
```
array(['Adelie'], dtype='<U9')
```

In [15]:
```python
# Probabilidades (softmax) que asigna el modelo a cada clase, para compararlas con las de Excel
pd.DataFrame(mlp.predict_proba(pd.DataFrame(nuevo_pinguino, columns=X.columns)), columns=mlp.classes_)
```

Out[15]:

	Adelie	Chinstrap	Gentoo
0	1.0	2.949251e-07	2.626329e-10

In [16]:
```python
# Guardar los pesos (coeficientes) de las capas del modelo MLP en archivos CSV
for i, coef in enumerate(mlp.coefs_):
    np.savetxt(f"pesos_capa_{i+1}.csv", coef, delimiter=",")

# Guardar los sesgos (bias) de las capas del modelo MLP en archivos CSV
for i, intercept in enumerate(mlp.intercepts_):
    np.savetxt(f"sesgos_capa_{i+1}.csv", intercept, delimiter=",")

# Esto generará archivos pesos_capa_1.csv, pesos_capa_2.csv, etc., que contienen los pesos de cada capa del modelo.
# Y archivos sesgos_capa_1.csv, sesgos_capa_2.csv, etc., que contienen los sesgos de cada capa del modelo.

# Nota 1: Asegúrate de que el modelo 'mlp' ya ha sido entrenado antes de ejecutar este código.
# Nota 2: MLPClassifier usa ReLU en las capas ocultas y sigmoid en la salida (para clases binarias) ó softmax en la salida (si ti
# Nota 3: El orden de los logits de salida corresponde al orden alfabético de las clases en y.
```

Entregar el archivo `.pdf` generado de este notebook:

1. Usa los pesos generados por el `MLPClassifier` para replicar la inferencia del modelo con el nuevo pinguino en Excel.

Inferencia en Excel (archivo `Inferencia_MLP_Excel.xlsx`). Los pesos y sesgos exportados (`pesos_capa_*.csv`, `sesgos_capa_*.csv`) se cargaron en las hojas `W1`, `b1`, `W2`, `b2`, `W3`, `b3`, y en la hoja `Inferencia` se replica el *forward pass*:

1. **Capa oculta 1:** $z_j^{(1)} = \sum_i x_i W_{ij}^{(1)} + b_j^{(1)}$ → `SUMAPRODUCTO` + `INDICE`; luego $h_j^{(1)} = \max(0, z_j^{(1)})$ → `MAX(0; …)` (ReLU).
2. **Capa oculta 2:** $z_j^{(2)} = \sum_i h_i^{(1)} W_{ij}^{(2)} + b_j^{(2)}$ y ReLU.
3. **Capa de salida:** $z_k^{(3)} = \sum_i h_i^{(2)} W_{ik}^{(3)} + b_k^{(3)}$ y **softmax** $p_k = e^{z_k} / \sum_m e^{z_m}$.
4. **Predicción:** la clase con mayor probabilidad (`INDICE` + `COINCIDIR` + `MAX`). El orden de las clases es alfabético: Adelie, Chinstrap, Gentoo.

Vista de valores (incluye el texto de cada fórmula con `FORMULATEXT`):

Inferencia del MLPClassifier (6 → 100 ReLU → 100 ReLU → 3 Softmax) replicada en Excel

Variable (x)	Valor	Nota
island	1	codificada con LabelEncoder
culmen_length_mm	45	
culmen_depth_mm	58	
flipper_length_mm	11	
body_mass_g	22	
sex	0	0 = FEMALE

k	Clase	z3 (logit)	Softmax P(clase)
1	Adelie	17,2810	1,000000
2	Chinstrap	2,2444	0,000000
3	Gentoo	-4,7793	0,000000

Especie predicha (Excel)	**Adelie**
Predicción Python (.predict)	Adelie
¿Coinciden?	Sí

Paso	Fórmula usada (FORMULATEXT)
z1 capa oculta 1 (B13)	=SUMAPRODUCTO($B$4:$B$9;INDICE('W1'!$A$1:$CV$6;0;$A13))+INDICE('b1'!$A$1:$A$100;$A13)
ReLU capa 1 (C13)	=MAX(0;B13)
z2 capa oculta 2 (D13)	=SUMAPRODUCTO($C$13:$C$112;INDICE('W2'!$A$1:$CV$100;0;$A13))+INDICE('b2'!$A$1:$A$100;$A13)
ReLU capa 2 (E13)	=MAX(0;D13)
z3 salida (G4)	=SUMAPRODUCTO($E$13:$E$112;INDICE('W3'!$A$1:$C$100;0;E4))+INDICE('b3'!$A$1:$A$3;E4)
Softmax (H4)	=EXP(G4)/SUMAPRODUCTO(EXP($G$4:$G$6))
Predicción (G8)	=INDICE(F4:F6;COINCIDIR(MAX(H4:H6);H4:H6;0))

Neurona j	z1 = x·W1[:,j] + b1[j]	h1 = ReLU(z1)	z2 = h1·W2[:,j] + b2[j]	h2 = ReLU(z2)
1	-11,8239	0,0000	-5,9814	0,0000
2	-3,6700	0,0000	-1,5069	0,0000
3	-7,3259	0,0000	-0,3334	0,0000
4	13,8803	13,8803	1,3896	1,3896
5	10,9087	10,9087	-1,7778	0,0000
6	-14,6138	0,0000	-2,2932	0,0000
7	-8,1084	0,0000	5,0913	5,0913
8	5,3773	5,3773	-14,0648	0,0000
9	-10,8134	0,0000	4,5449	4,5449
10	-14,7009	0,0000	-3,1543	0,0000
11	-3,1049	0,0000	-0,8961	0,0000
12	2,9947	2,9947	-2,5930	0,0000

Vista de fórmulas, capa oculta 1:

Variable (x)	Valor
island	1
culmen_length_mm	45
culmen_depth_mm	58
flipper_length_mm	11
body_mass_g	22
sex	0

Neurona j	z1 = x·W1[:,j] + b1[j]
1	=SUMAPRODUCTO($B$4:$B$9;INDICE('W1'!$A$1:$CV$6;0;$A13))+INDICE('b1'!$A$1:$A$100;$A13)
2	=SUMAPRODUCTO($B$4:$B$9;INDICE('W1'!$A$1:$CV$6;0;$A14))+INDICE('b1'!$A$1:$A$100;$A14)
3	=SUMAPRODUCTO($B$4:$B$9;INDICE('W1'!$A$1:$CV$6;0;$A15))+INDICE('b1'!$A$1:$A$100;$A15)
4	=SUMAPRODUCTO($B$4:$B$9;INDICE('W1'!$A$1:$CV$6;0;$A16))+INDICE('b1'!$A$1:$A$100;$A16)

Vista de fórmulas, ReLU y capa oculta 2:

h1 = ReLU(z1)	z2 = h1·W2[:,j] + b2[j]	h2 = ReLU(z2)
=MAX(0;B13)	=SUMAPRODUCTO($C$13:$C$112;INDICE('W2'!$A$1:$CV$100;0;$A13))+INDICE('b2'!$A$1:$A$100;$A13)	=MAX(0;D13)
=MAX(0;B14)	=SUMAPRODUCTO($C$13:$C$112;INDICE('W2'!$A$1:$CV$100;0;$A14))+INDICE('b2'!$A$1:$A$100;$A14)	=MAX(0;D14)
=MAX(0;B15)	=SUMAPRODUCTO($C$13:$C$112;INDICE('W2'!$A$1:$CV$100;0;$A15))+INDICE('b2'!$A$1:$A$100;$A15)	=MAX(0;D15)
=MAX(0;B16)	=SUMAPRODUCTO($C$13:$C$112;INDICE('W2'!$A$1:$CV$100;0;$A16))+INDICE('b2'!$A$1:$A$100;$A16)	=MAX(0;D16)

Vista de fórmulas, capa de salida, softmax y predicción:

Clase	z3 (logit)	Softmax P(clase)
Adelie	=SUMAPRODUCTO($E$13:$E$112;INDICE('W3'!$A$1:$C$100;0;E4))+INDICE('b3'!$A$1:$A$3;E4)	=EXP(G4)/SUMAPRODUCTO(EXP($G$4:$G$6))
Chinstrap	=SUMAPRODUCTO($E$13:$E$112;INDICE('W3'!$A$1:$C$100;0;E5))+INDICE('b3'!$A$1:$A$3;E5)	=EXP(G5)/SUMAPRODUCTO(EXP($G$4:$G$6))
Gentoo	=SUMAPRODUCTO($E$13:$E$112;INDICE('W3'!$A$1:$C$100;0;E6))+INDICE('b3'!$A$1:$A$3;E6)	=EXP(G6)/SUMAPRODUCTO(EXP($G$4:$G$6))
Especie predicha (Excel)	**=INDICE(F4:F6;COINCIDIR(MAX(H4:H6);H4:H6;0))**	
Predicción Python (.predict)	Adelie	
¿Coinciden?	=SI(G8=G9;"Sí";"No")	

2. Responde a la siguiente pregunta: ¿Cuál es la especie predicha para el nuevo pinguino usando el método `.predict`?

Respuesta: con el método `.predict`, el modelo clasifica al nuevo pingüino `[[1, 45, 58, 11, 22, 0]]` como **Adelie**. Según `predict_proba`, la probabilidad es ≈ 1.0 para Adelie, 2.95×10^{-7} para Chinstrap y 2.63×10^{-10} para Gentoo.

In [14]:
```
# Hacemos una prueba de predicción de especie con valores de un nuevo pinguino (no visto por el modelo)
# Creamos una lista con los valores de las variables predictoras del nuevo pinguino
# Puedes usar estos valores de ejemplo o cambiarlos por los valores aleatorios de un nuevo pinguino:
#   island = 1 (Biscoe)
#   culmen_length_mm = 45
#   culmen_depth_mm = 58
#   flipper_length_mm = 11
#   body_mass_g = 22
#   sex = 0 (FEMALE)
# Guardamos la lista en la variable 'nuevo_pinguino'
nuevo_pinguino = [[1, 45, 58, 11, 22, 0]]

# Hacemos una predicción de la especie del 'nuevo_pinguino' con el método 'predict' del modelo 'mlp'
# Guardamos la predicción en la variable 'nueva_prediccion'
nueva_prediccion = mlp.predict(nuevo_pinguino)
nueva_prediccion
```

Out[14]:
```
array(['Adelie'], dtype='<U9')
```

In [15]:
```
# Probabilidades (softmax) que asigna el modelo a cada clase, para compararlas con las de Excel
pd.DataFrame(mlp.predict_proba(pd.DataFrame(nuevo_pinguino, columns=X.columns)), columns=mlp.classes_)
```

Out[15]:

	Adelie	Chinstrap	Gentoo
0	1.0	2.949251e-07	2.626329e-10

3. Responde a la siguiente pregunta: ¿Cuál es la especie predicha para el nuevo pinguino usando Excel?

Respuesta: en Excel la especie predicha también es **Adelie**. Los logits de salida son $z^{(3)}$ = (17.2810, 2.2444, −4.7793) y el softmax da P(Adelie) = 0.9999997, P(Chinstrap) = 2.95×10^{-7} y P(Gentoo) = 2.63×10^{-10}. Son las mismas probabilidades de `predict_proba`, así que la inferencia manual en Excel reproduce exactamente el *forward pass* del `MLPClassifier`.

Clase	z3 (logit)	Softmax P(clase)
Adelie	=SUMAPRODUCTO($E$13:$E$112;INDICE('W3'!$A$1:$C$100;0;E4))+INDICE('b3'!$A$1:$A$3;E4)	=EXP(G4)/SUMAPRODUCTO(EXP($G$4:$G$6))
Chinstrap	=SUMAPRODUCTO($E$13:$E$112;INDICE('W3'!$A$1:$C$100;0;E5))+INDICE('b3'!$A$1:$A$3;E5)	=EXP(G5)/SUMAPRODUCTO(EXP($G$4:$G$6))
Gentoo	=SUMAPRODUCTO($E$13:$E$112;INDICE('W3'!$A$1:$C$100;0;E6))+INDICE('b3'!$A$1:$A$3;E6)	=EXP(G6)/SUMAPRODUCTO(EXP($G$4:$G$6))
Especie predicha (Excel)	**=INDICE(F4:F6;COINCIDIR(MAX(H4:H6);H4:H6;0))**	
Predicción Python (.predict)	Adelie	
¿Coinciden?	=SI(G8=G9;"Sí";"No")	

Inferencia del MLPClassifier (6 → 100 ReLU → 100 ReLU → 3 Softmax) replicada en Excel

Variable (x)	Valor	Nota
island	1	codificada con LabelEncoder
culmen_length_mm	45	
culmen_depth_mm	58	
flipper_length_mm	11	
body_mass_g	22	
sex	0	0 = FEMALE

k	Clase	z3 (logit)	Softmax P(clase)
1	Adelie	17,2810	1,000000
2	Chinstrap	2,2444	0,000000
3	Gentoo	-4,7793	0,000000
	Especie predicha (Excel)	**Adelie**	
	Predicción Python (.predict)	Adelie	
	¿Coinciden?	Sí	

Paso	Fórmula usada (FORMULATEXT)
z1 capa oculta 1 (B13)	=SUMAPRODUCTO($B$4:$B$9;INDICE('W1'!$A$1:$CV$6;0;$A13))+INDICE('b1'!$A$1:$A$100;$A13)
ReLU capa 1 (C13)	=MAX(0;B13)
z2 capa oculta 2 (D13)	=SUMAPRODUCTO($C$13:$C$112;INDICE('W2'!$A$1:$CV$100;0;$A13))+INDICE('b2'!$A$1:$A$100;$A13)
ReLU capa 2 (E13)	=MAX(0;D13)
z3 salida (G4)	=SUMAPRODUCTO($E$13:$E$112;INDICE('W3'!$A$1:$C$100;0;E4))+INDICE('b3'!$A$1:$A$3;E4)
Softmax (H4)	=EXP(G4)/SUMAPRODUCTO(EXP($G$4:$G$6))
Predicción (G8)	=INDICE(F4:F6;COINCIDIR(MAX(H4:H6);H4:H6;0))

Neurona j	z1 = x·W1[:,j] + b1[j]	h1 = ReLU(z1)	z2 = h1·W2[:,j] + b2[j]	h2 = ReLU(z2)
1	-11,8239	0,0000	-5,9814	0,0000
2	-3,6700	0,0000	-1,5069	0,0000
3	-7,3259	0,0000	-0,3334	0,0000
4	13,8803	13,8803	1,3896	1,3896
5	10,9087	10,9087	-1,7778	0,0000
6	-14,6138	0,0000	-2,2932	0,0000
7	-8,1084	0,0000	5,0913	5,0913
8	5,3773	5,3773	-14,0648	0,0000
9	-10,8134	0,0000	4,5449	4,5449
10	-14,7009	0,0000	-3,1543	0,0000
11	-3,1049	0,0000	-0,8961	0,0000
12	2,9947	2,9947	-2,5930	0,0000

Useful Resources:

- https://support.microsoft.com/es-es/office/función-mmult-40593ed7-a3cd-4b6b-b9a3-e4ad3c7245eb

Prepared by:

Manuel Eugenio Morocho-Cayamcela